



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

INSTITUTO DE INVESTIGACIONES EN MATEMÁTICAS
APLICADAS Y EN SISTEMAS (IIMAS)

Desarrollo de Aplicaciones Web Desacopladas

Documentación de aplicación web para cuestionarios (Proyecto Final)

Equipo:

Alejandro Iram Ramírez Nava

Erick José Fabián Sandoval

Abril Minerva Estrada Montaña

Licenciatura en Ciencia de Datos

Profesor: Israel Sandoval Grajeda

3 de junio de 2026

Repositorio GitHub:

<https://github.com/0ElaLe/AplicacionCuestionarios>

Índice

1. Modelo del Sistema	2
1.1. Componentes y características	2
1.2. Relaciones	2
1.3. Diagrama Entidad-Relación	3
1.4. Experimentos	3
2. Stack Tecnológico	3
2.1. Backend Flask	4
2.2. Frontend Vue 3	4
3. Algoritmo	4
3.1. Diagrama de procesos con carriles FRONT / BACK	4
3.2. Flujo de autenticación	5
3.3. Flujo de creación de cuestionario	6
3.4. Flujo de respuesta a cuestionario	7
4. Diseño de la API — Endpoints	7
4.1. Autenticación — /api/auth	8
4.2. Formularios — /api/v2/formularios	9
4.3. Respuestas y avance parcial	10
4.4. Endpoints de apoyo	10
4.5. Ejemplo de petición	10
5. Manual de Instalación y Ejecución	11
5.1. Paso 1 — Clonar el repositorio	11
5.2. Paso 2 — Configurar el backend	11
5.3. Paso 3 — Iniciar el backend	11
5.4. Paso 4 — Configurar e iniciar el frontend	11
5.5. Resumen de comandos	12
6. Manual de Uso de la Aplicación	12
6.1. Pantalla de inicio de sesión	12
6.2. Registro de usuario	13
6.3. Panel principal (Dashboard)	13
6.4. Crear cuestionario	14
6.5. Contestar cuestionario — Identificación	15
6.6. Contestar cuestionario — Preguntas	15
6.7. Editar cuestionario	16
6.8. Ver resultados	16
6.9. Confirmación de envío	17
7. Conclusión	17

1. Modelo del Sistema

1.1. Componentes y características

El sistema es una aplicación web desacoplada para la creación, distribución y análisis de cuestionarios en línea. Los componentes principales son:

Los formularios (cuestionarios): Cada cuestionario tiene un título, descripción opcional, visibilidad (público o privado), estado (publicado o archivado) y un código único de 10 caracteres para compartirlo. El creador puede editar el cuestionario mientras no tenga respuestas y puede archivar/publicarlo en cualquier momento.

Las preguntas: Pertenecen a un formulario y tienen un tipo que determina cómo el respondente interactuará. El sistema implementa siete tipos de pregunta:

1. **Texto corto:** respuesta textual breve.
2. **Texto largo:** respuesta textual extensa (párrafo).
3. **Opción única:** selección de una opción predefinida (radio button).
4. **Opción múltiple:** selección de una o varias opciones (checkboxes).
5. **Escala 1–5:** valor numérico discreto en ese rango.
6. **Sí / No:** caso especial de opción única con dos opciones predefinidas.
7. **Fecha:** selector de fecha (día/mes/año).

Los usuarios: Existen dos roles: *creador* (registrado con contraseña, puede crear/editar/archivar cuestionarios y ver resultados) y *respondente* (puede ser anónimo, se identifica con nombre y correo para responder). El sistema detecta si un usuario ya respondió un cuestionario y bloquea el reenvío.

Las respuestas: Se almacenan de forma atómica dentro de una transacción. Dependiendo del tipo de pregunta, la respuesta se guarda como texto, número o referencia a opciones predefinidas. El sistema también guarda avances parciales (guardado automático) mientras el respondente contesta.

1.2. Relaciones

El modelo relacional se basa en las siguientes entidades y relaciones:

- **Usuario – Formulario (N:M):** Un usuario crea muchos formularios; un formulario tiene un creador. La relación se materializa a través de la entidad `intentos_formulario`.
- **Formulario – Pregunta (1:N):** Un formulario contiene muchas preguntas; cada pregunta pertenece a un único formulario.
- **Pregunta – TipoPregunta (N:1):** Cada pregunta tiene un tipo del catálogo.
- **Pregunta – OpcionRespuesta (1:N):** Las preguntas de opción tienen opciones predefinidas.
- **IntentosFormulario – Respuesta (1:N):** Un intento contiene múltiples respuestas (una por pregunta).

- **Respuesta – RespuestaOpcion (1:N):** Las respuestas de selección registran las opciones elegidas.

1.3. Diagrama Entidad-Relación

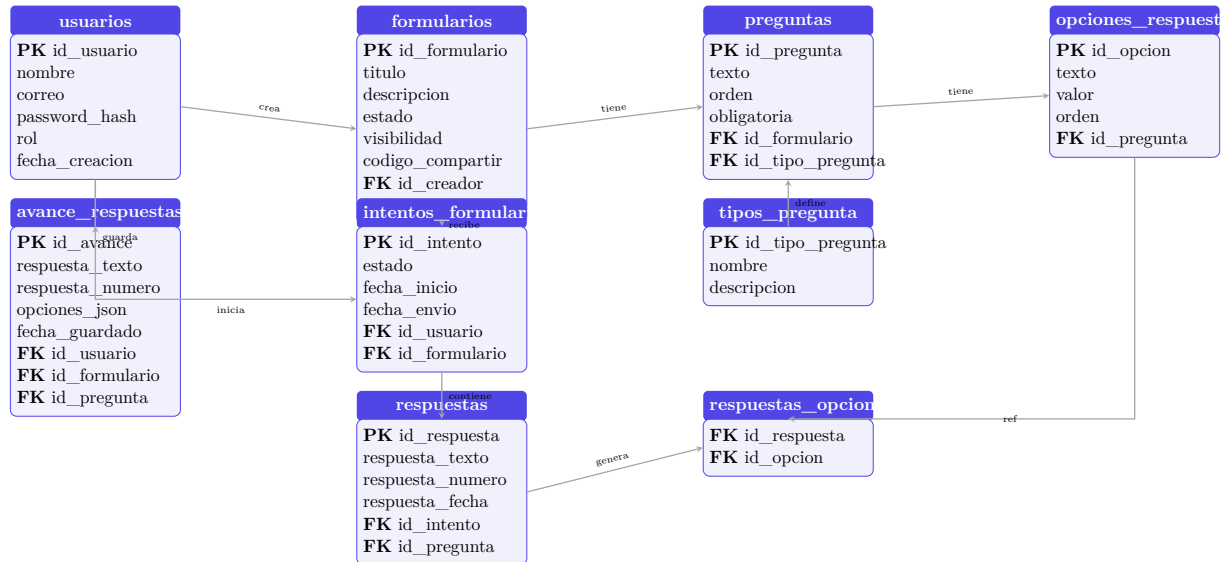


Figura 1: Diagrama Entidad-Relación — tablas de la base de datos SQLite.

1.4. Experimentos

1. El sistema despliega las preguntas al usuario.
2. El sistema identifica a cada usuario para relacionarlo con sus respuestas.
3. El sistema es capaz de almacenar las respuestas de cada usuario.
4. El sistema identifica si el usuario ya contestó el formulario.
5. El usuario es capaz de ver sus respuestas.

2. Stack Tecnológico

El sistema adopta una arquitectura **cliente-servidor desacoplada**. El backend expone una API REST que el frontend consume de forma independiente.



Figura 2: Arquitectura cliente-servidor desacoplada.

2.1. Backend Flask

El archivo `backend/app.py` crea la aplicación Flask, habilita CORS y registra los blueprints. La lógica está distribuida en:

- `modelo.py` — acceso a datos (SQLite), lógica de negocio.
- `v2_routes.py` — rutas v2 (formularios dinámicos, auth, resultados).
- `auth_routes.py` — registro e inicio de sesión.
- `rutas.py` — rutas legacy (flujo sin autenticación).
- `middleware.py` — validación de token Bearer.

2.2. Frontend Vue 3

El frontend usa Vue 3 con Composition API, TypeScript y Vite. Está organizado en:

- `src/views/` — vistas principales (Dashboard, Crear, Editar, Contestar, Resultados, Login, Registro).
- `src/components/questions/` — componentes de pregunta por tipo.
- `src/services/api.service.ts` — capa de comunicación con la API.
- `src/composables/` — lógica reutilizable (`useAuth`, `useApp`).
- `src/router/` — enrutamiento con guards de autenticación.

3. Algoritmo

Capa	Tecnología	Versión
Frontend	Vue 3 (Composition API + TypeScript)	3.x
Build tool	Vite	6.x
Routing (FE)	Vue Router	4.x
Backend	Python + Flask	3.x
CORS	Flask-CORS	4.x
Base de datos	SQLite (módulo <code>sqlite3</code>)	nativa
Formato de datos	JSON	—
Auth	Token Bearer (SHA-256 + salt)	—

Cuadro 1: Stack tecnológico del sistema.

3.1. Diagrama de procesos con carriles FRONT / BACK

Convención: *Solicitar* es una llamada del frontend a la API. *Entregar* y *Guardar* son

operaciones del backend. *Desplegar* significa mostrar una pantalla o componente en el frontend.

Flujo de respuesta a cuestionario

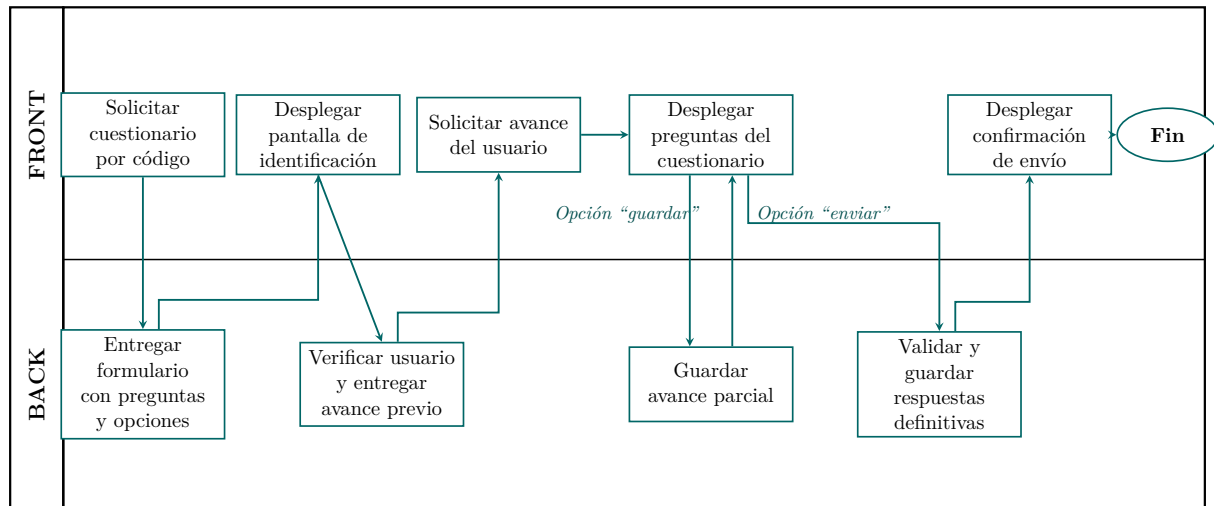


Figura 3: Diagrama de procesos con carriles FRONT/BACK — flujo de respuesta a cuestionario.

Se presentan los tres flujos principales de la aplicación.

3.2. Flujo de autenticación

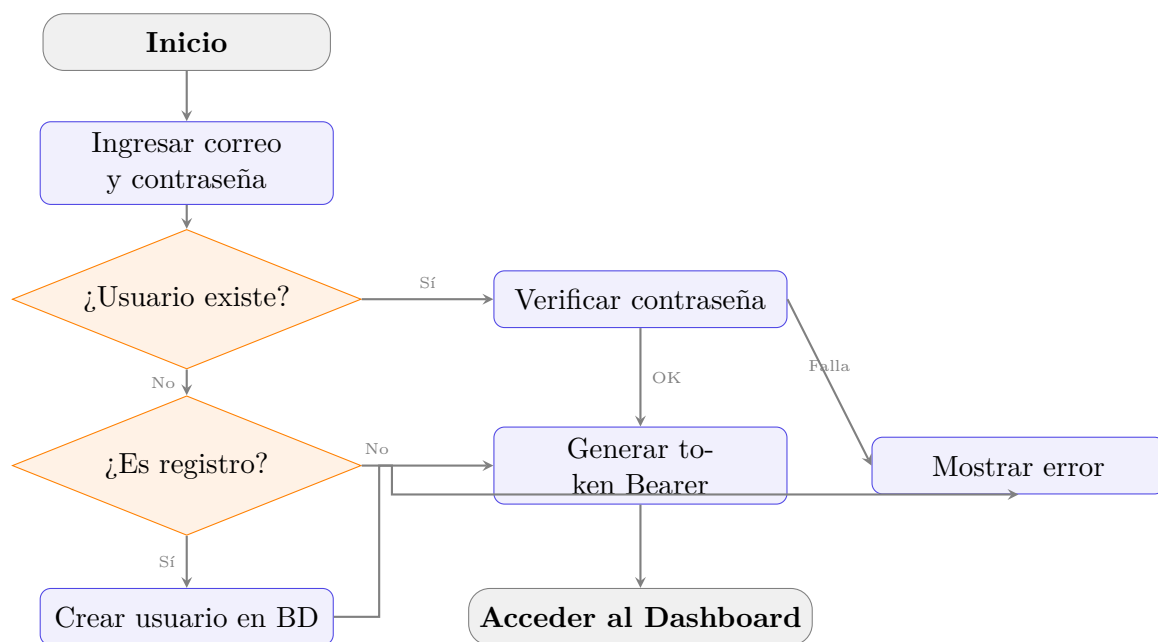


Figura 4: Flujo de autenticación (registro e inicio de sesión).

3.3. Flujo de creación de cuestionario

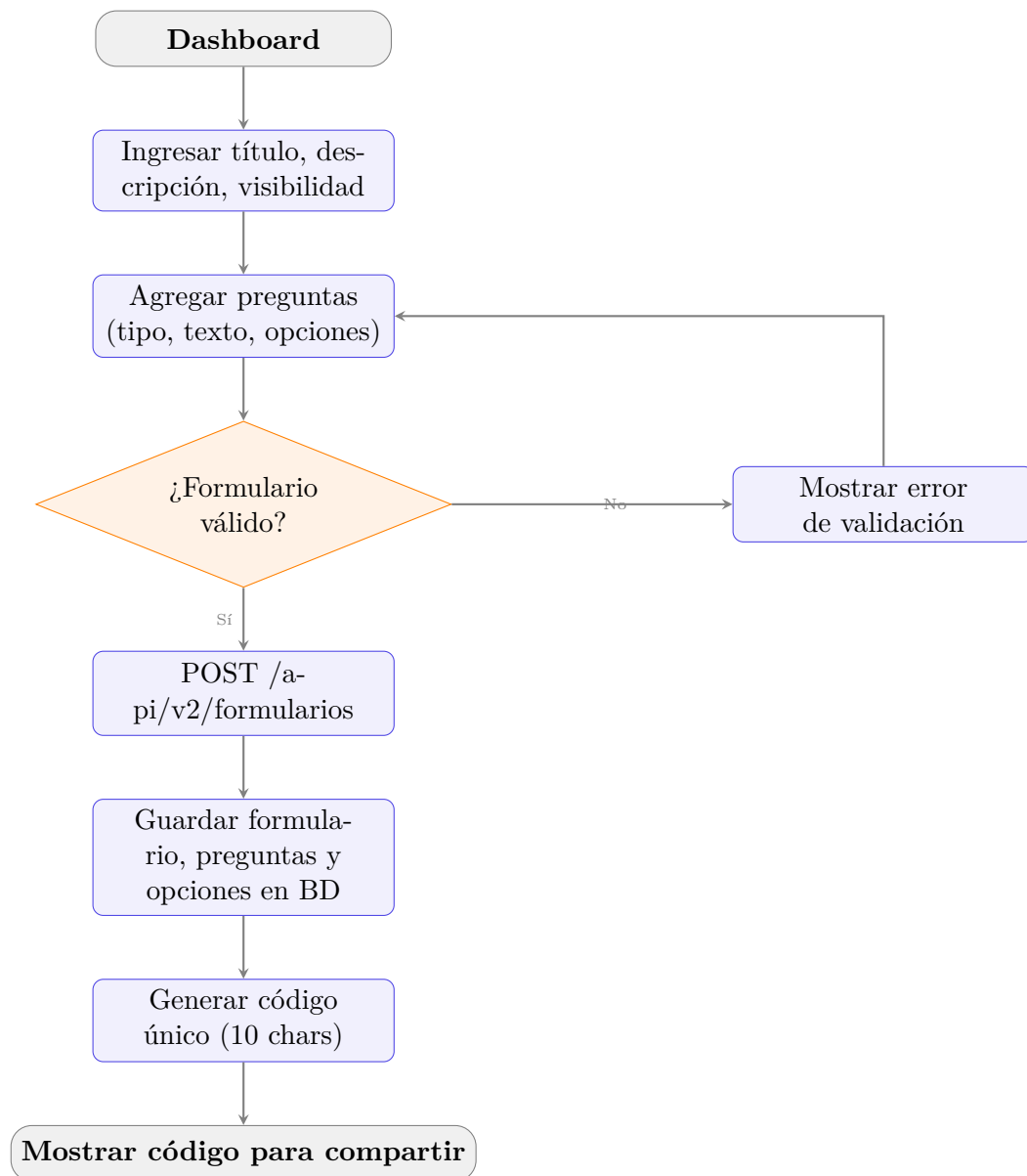


Figura 5: Flujo de creación de un cuestionario.

3.4. Flujo de respuesta a cuestionario

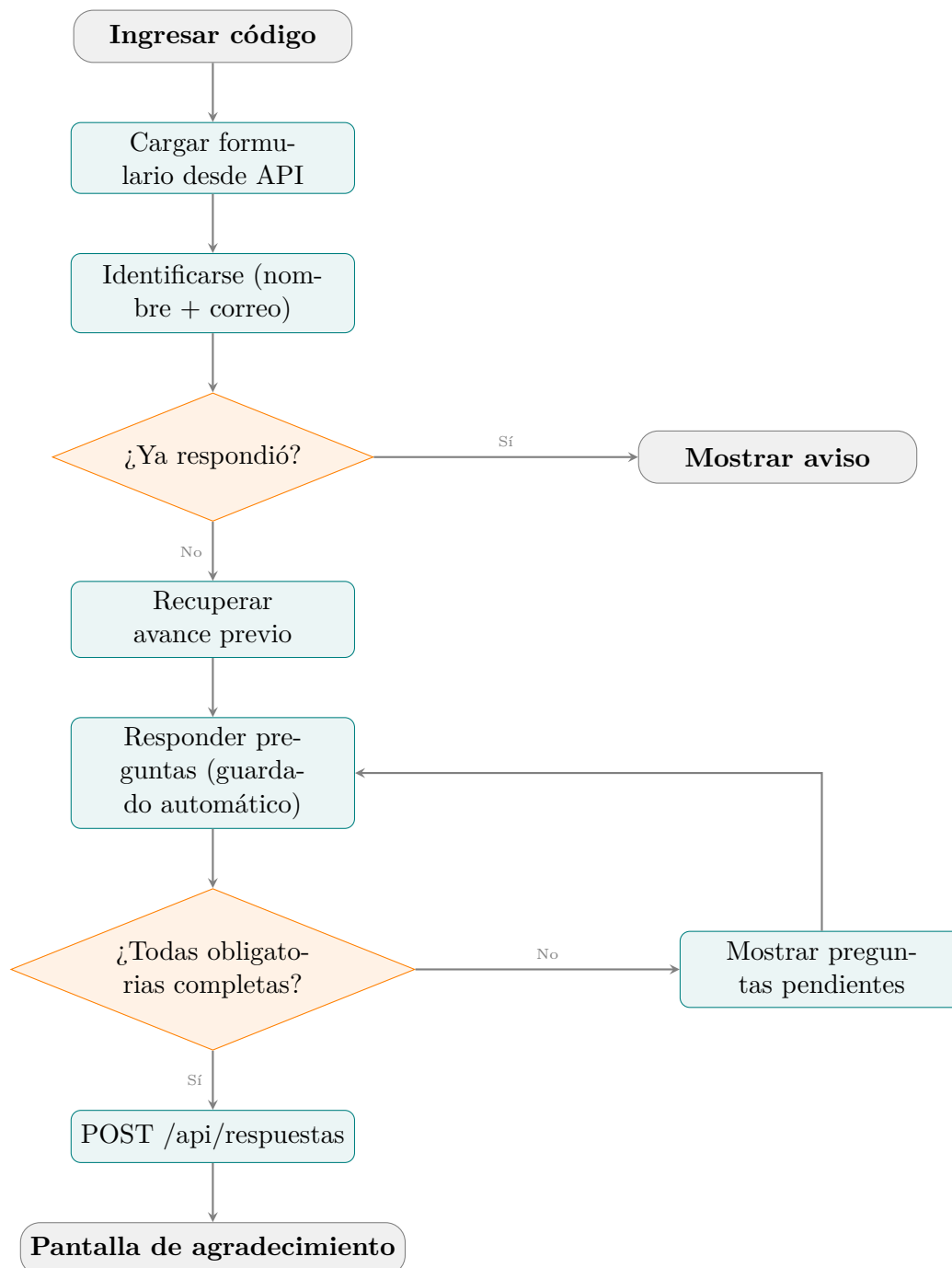


Figura 6: Flujo de respuesta a un cuestionario.

4. Diseño de la API — Endpoints

Todos los endpoints devuelven JSON. Los marcados con `requieren` la cabecera **Authorization: Bearer <token>**.

4.1. Autenticación — /api/auth

Método	Ruta	Entrada	Salida
POST	/api/auth/registro	nombre, correo, password	{ token, usuario }
POST	/api/auth/login	correo, password	{ token, usuario }
GET	/api/auth/me	—	{ usuario }

Cuadro 2: Endpoints de autenticación.

4.2. Formularios — /api/v2/formularios

Método	Ruta	Entrada	Salida
POST	/api/v2/formularios	titulo, descripcion, visibilidad, preguntas[]	{ id_formulario, codigo_compartir }
GET	/api/v2/formularios/mios	—	{ formularios[] }
GET	/api/v2/formularios/publicos	—	{ formularios[] }
GET	/api/v2/formularios/ codigo/:codigo	Código en URL	{ formulario } con preguntas y opciones
GET	/api/v2/formularios/:id	id en URL	{ formulario } con preguntas y opciones
PUT	/api/v2/formularios/:id	titulo, descripcion, visibilidad, preguntas[]	{ preguntas_editadas, tiene_respuestas }
PATCH	/api/v2/formularios/ :id/archivar	—	{ estado }
GET	/api/v2/formularios/ :id/resultados	id en URL	{ total_respuestas, preguntas[] }
GET	/api/v2/formularios/ respondidos/:id_usuario	id_usuario URL	en { formularios[] }

Cuadro 3: Endpoints de formularios (v2). Los marcados con — son solo para el creador.

4.3. Respuestas y avance parcial

Método	Ruta	Entrada	Salida
POST	/api/respuestas	id_usuario, id_formulario, respuestas[]	{ mensaje, id_intento }
POST	/api/v2/avance	id_usuario, id_formulario, respuestas[]	{ mensaje, ya_enviado }
GET	/api/v2/avance/ :id_usuario/:id_formulario	IDs en URL	{ estado, respuestas[] }

Cuadro 4: Endpoints de envío de respuestas y guardado automático.

4.4. Endpoints de apoyo

Método	Ruta	Entrada	Salida
GET	/api/health	—	{ status, mensaje }
POST	/api/usuarios	nombre, correo	{ mensaje, usuario }
GET	/api/usuarios/:id/ respondio/:id_form	IDs en URL	{ respondio: bool }

Cuadro 5: Endpoints de apoyo para el flujo de respondente anónimo.

4.5. Ejemplo de petición

Listing 1: Body JSON para POST /api/respuestas

```
{
  "id_usuario": 3,
  "id_formulario": 7,
  "respuestas": [
    { "id_pregunta": 10, "respuesta_texto": "Juan Perez" },
    { "id_pregunta": 11, "respuesta_texto": "2003-05-14" },
    { "id_pregunta": 12, "opciones": [45] },
    { "id_pregunta": 13, "opciones": [48, 50] },
    { "id_pregunta": 14, "respuesta_numero": 4 }
  ]
}
```

5. Manual de Instalación y Ejecución

Repositorio: <https://github.com/0ElaLe/AplicacionCuestionarios>

Requisitos previos: Python 3.10+, Node.js 18+, Git, VS Code.

5.1. Paso 1 — Clonar el repositorio

Abrir la terminal integrada de VS Code (Ctrl+`) y ejecutar:

```
git clone https://github.com/0ElaLe/AplicacionCuestionarios.git
cd AplicacionCuestionarios
```

5.2. Paso 2 — Configurar el backend

```
# Crear entorno virtual
python -m venv .venv

# Activar el entorno virtual (Windows PowerShell)
.venv\Scripts\activate

# Instalar dependencias
pip install -r backend\requirements.txt

# Ejecutar migración de base de datos (solo la primera vez)
python backend\migrate_v2.py
```

5.3. Paso 3 — Iniciar el backend

```
# Con el entorno virtual activo
python backend\app.py
```

La API quedará disponible en <http://localhost:5000>. Se verá en consola:

```
* Running on http://127.0.0.1:5000
* Debug mode: on
```

5.4. Paso 4 — Configurar e iniciar el frontend

Abrir una segunda terminal en VS Code (Ctrl+Shift+5 o botón +):

```
cd frontend-vue
npm install
npm run dev
```

El frontend quedará disponible en <http://localhost:5173> (Vite mostrará la URL exacta).

5.5. Resumen de comandos

Terminal	Comando	Resultado
1 — Backend	<code>.venv\Scripts\activate</code>	Activa entorno Python
1 — Backend	<code>python backend\migrate_v2.py</code>	Migración (solo 1ª vez)
1 — Backend	<code>python backend\app.py</code>	API en puerto 5000
2 — Frontend	<code>npm install</code> (solo 1ª vez)	Instala dependencias
2 — Frontend	<code>npm run dev</code>	App en puerto 5173

Cuadro 6: Comandos para iniciar la aplicación desde cero.

6. Manual de Uso de la Aplicación

6.1. Pantalla de inicio de sesión

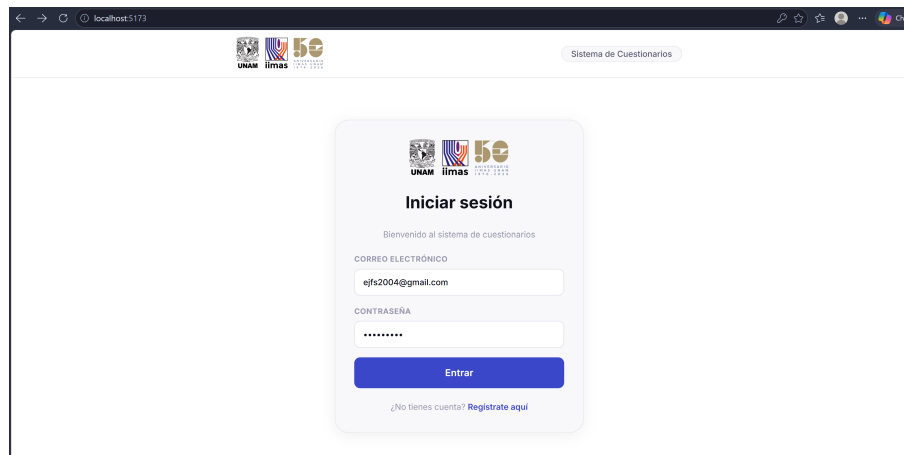


Figura 7: Pantalla de inicio de sesión. El usuario ingresa correo y contraseña.

Al ingresar a <http://localhost:5173> se presenta la pantalla de login. El usuario introduce su correo electrónico y contraseña. Si no tiene cuenta, puede hacer clic en el enlace *Crear cuenta*.

6.2. Registro de usuario

Figura 8: Pantalla de registro de nuevo usuario.

El formulario de registro solicita nombre completo, correo electrónico y contraseña (mínimo 6 caracteres). Al registrarse correctamente, el sistema inicia sesión automáticamente y redirige al dashboard.

6.3. Panel principal (Dashboard)

Figura 9: Panel principal con las opciones de crear cuestionario, contestar por código y las pestañas de listas.

El dashboard muestra:

- **Crear cuestionario:** botón para ir al formulario de creación.
- **Contestar cuestionario:** campo para ingresar el código de 10 caracteres.
- **Mis creados:** lista de cuestionarios creados por el usuario con acciones de resultados (), editar () y archivar/publicar (/).

- **Historial:** cuestionarios que el usuario ya respondió.
- **Explorar públicos:** todos los cuestionarios públicos disponibles.

6.4. Crear cuestionario

Figura 10: Vista de creación de cuestionario con tipos de pregunta, botones de reordenar y previsualización.

El creador puede:

- Escribir título, descripción y seleccionar visibilidad (público / privado).
- Agregar preguntas de 7 tipos: texto corto, texto largo, opción única, opción múltiple, escala 1–5, Sí/No y fecha.
- Reordenar preguntas con los botones ↑ y ↓.
- Para los tipos Sí/No, escala y fecha se muestra una previsualización; para opción única/múltiple se agregan las opciones manualmente.

6.5. Contestar cuestionario — Identificación

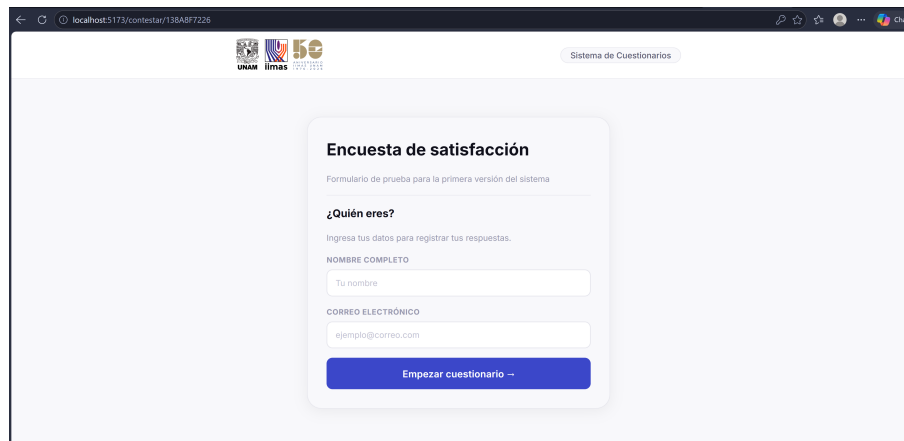
The screenshot shows a web browser window with the URL 'localhost:5173/contestar/138A87225'. The page features the UNAM and IIMAS logos at the top left and a 'Sistema de Cuestionarios' label at the top right. The main content is a light blue box titled 'Encuesta de satisfacción' with the subtitle 'Formulario de prueba para la primera versión del sistema'. It asks '¿Quién eres?' and prompts the user to 'Ingresa tus datos para registrar tus respuestas.' There are two input fields: 'NOMBRE COMPLETO' with the placeholder 'Tu nombre' and 'CORREO ELECTRÓNICO' with the placeholder 'ejemplo@correo.com'. A blue button at the bottom says 'Empezar cuestionario →'.

Figura 11: Pantalla de identificación del respondente antes de iniciar el cuestionario.

El respondente ingresa el código compartido y luego se identifica con su nombre y correo. El sistema verifica si ya contestó anteriormente y recupera el avance parcial si existe.

6.6. Contestar cuestionario — Preguntas

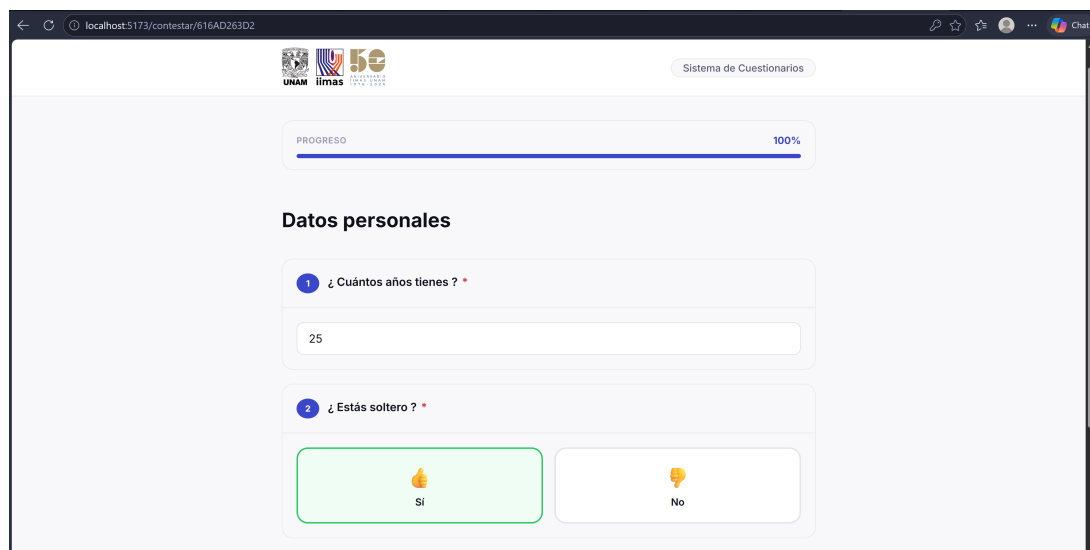
The screenshot shows a web browser window with the URL 'localhost:5173/contestar/616AD263D2'. The page features the UNAM and IIMAS logos at the top left and a 'Sistema de Cuestionarios' label at the top right. A progress bar at the top indicates 'PROGRESO' at '100%'. Below it, the section 'Datos personales' contains two questions. Question 1 is '¿ Cuántos años tienes ? *' with a text input field containing '25'. Question 2 is '¿ Estás soltero ? *' with two radio button options: 'Sí' (highlighted with a green border) and 'No'.

Figura 12: Vista de respuesta con barra de progreso y distintos tipos de pregunta.

La vista de respuesta incluye:

- Barra de progreso que indica el porcentaje de preguntas respondidas.
- Guardado automático al cambiar de pregunta (y al cerrar la pestaña).
- Los botones / para preguntas Sí/No.
- Selector de fecha nativo del navegador para preguntas de tipo fecha.
- Botón *Enviar respuestas* que valida las preguntas obligatorias antes de guardar.

6.7. Editar cuestionario

Figura 13: Vista de edición. Si ya existen respuestas, solo se pueden editar los metadatos.

Accesible desde el botón del dashboard. Permite modificar título, descripción y visibilidad en cualquier momento. La edición de preguntas solo está disponible si el cuestionario no tiene respuestas todavía (el sistema muestra un aviso en caso contrario).

6.8. Ver resultados

Figura 14: Vista de resultados con estadísticas por pregunta: barras CSS, promedio de escala y listado de respuestas textuales.

Accesible desde el botón del dashboard (solo para el creador). Muestra:

- Total de respuestas recibidas.
- **Preguntas de texto/fecha:** listado de las respuestas enviadas.
- **Escala:** promedio numérico y distribución por valor (1–5) con barras CSS.

- **Opción única/múltiple/Sí-No:** conteo y porcentaje por opción con barras de progreso (verde para Sí, rojo para No).

6.9. Confirmación de envío

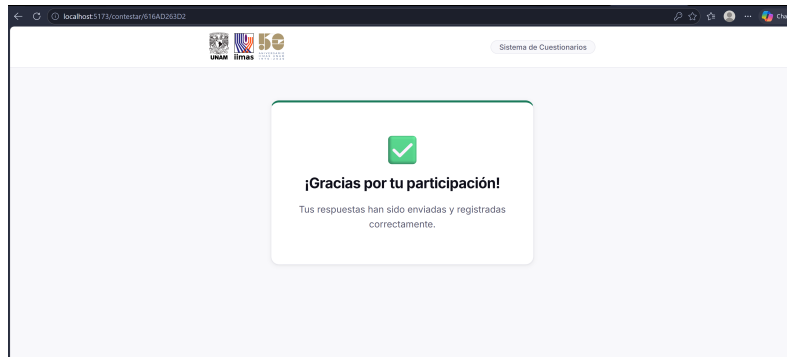


Figura 15: Pantalla de agradecimiento tras enviar correctamente las respuestas.

Tras enviar el cuestionario, el sistema muestra una pantalla de confirmación. El intento queda registrado y el sistema impide que el mismo usuario responda de nuevo.

7. Conclusión

El proyecto implementa una aplicación web desacoplada completa para la creación y gestión de cuestionarios en línea. El backend Flask expone una API RESTful con autenticación por token, y el frontend Vue 3 consume dicha API de forma independiente.

Las funcionalidades implementadas en esta versión final incluyen: registro e inicio de sesión con contraseña hasheada, creación de cuestionarios con 7 tipos de pregunta, reordenamiento de preguntas, edición de cuestionarios, archivado/publicación, vista de resultados con estadísticas agregadas, guardado automático de avance y diferenciación entre cuestionarios públicos y privados.

El código fuente, historial de cambios y este documento se encuentran disponibles en:

<https://github.com/0ElaLe/AplicacionCuestionarios>